```
    Supported options are:
      -h         This message
      ...
  USAGE
end
```

the user would see the usage message aligned against the left margin.

Technically, it looks for the least indented line in the whole string, and removes that amount of leading whitespace.

Defined in `active_support/core_ext/string/strip.rb`.

5.9 `indent`

Indents the lines in the receiver:

```
<<EOS.indent(2)
def some_method
  some_code
end
EOS
# =>
  def some_method
    some_code
  end
```

The second argument, `indent_string`, specifies which indent string to use. The default is `nil`, which tells the method to make an educated guess peeking at the first indented line, and fallback to a space if there is none.

```
"  foo".indent(2)        # => "    foo"
"foo\n\t\tbar".indent(2) # => "\t\tfoo\n\t\t\t\tbar"
"foo".indent(2, "\t")    # => "\t\tfoo"
```

While `indent_string` is typically one space or tab, it may be any string.

The third argument, `indent_empty_lines`, is a flag that says whether empty lines should be indented. Default is false.

```
"foo\n\nbar".indent(2)            # => "  foo\n\n  bar"
"foo\n\nbar".indent(2, nil, true) # => "  foo\n  \n  bar"
```

The `indent!` method performs indentation in-place.

Defined in `active_support/core_ext/string/indent.rb`.

5.10 Access

5.10.1 `at(position)`

Returns the character of the string at position `position`:

```
"hello".at(0)  # => "h"
"hello".at(4)  # => "o"
"hello".at(-1) # => "o"
"hello".at(10) # => nil
```

Defined in `active_support/core_ext/string/access.rb`.

5.10.2 `from(position)`

Returns the substring of the string starting at position `position`:

```
"hello".from(0)  # => "hello"
"hello".from(2)  # => "llo"
"hello".from(-2) # => "lo"
"hello".from(10) # => nil
```

Defined in `active_support/core_ext/string/access.rb`.